Nix 入門

aoki-taquan

2026 年 04 月 11 日

目次

1 はじめに 5
1.1 Nix とは 5
1.2 Nix エコシステムの全体像 5
1.3 他のパッケージマネージャとの比較 5
1.4 本書の対象読者 6
1.5 前提知識 6
2 インストール 7
2.1 システム要件 7
2.2 インストール方法 7
2.2.1 マルチユーザーインストール（推奨） 7
2.2.2 シングルユーザーインストール 7
2.3 インストールの確認 8
2.4 Nix の実験的機能を有効にする 8
2.5 アンインストール 8
2.5.1 Linux（マルチユーザー） 8
2.5.2 macOS 8
3 基本概念 9
3.1 Nix Store 9
3.1.1 Nix Store の特性 9
3.2 デリベーション（Derivation） 9
3.3 プロファイルと世代 10
3.4 チャネル 10
3.5 クロージャ（Closure） 11
4 パッケージ管理 12
4.1 パッケージの検索 12
4.1.1 コマンドラインでの検索 12
4.1.2 Web での検索 12
4.2 パッケージのインストール 12
4.2.1 命令的なインストール（nix-env） 12
4.2.2 一時的な利用（nix-shell / nix shell） 12
4.3 パッケージの削除 13
4.4 インストール済みパッケージの管理 13
4.5 宣言的なパッケージ管理 13
4.5.1 home-manager による管理 13
4.6 バイナリキャッシュ 14
5 Nix 言語基礎 15
5.1 Nix 言語の概要 15
5.2 基本的なデータ型 15
5.2.1 数値と文字列 15

5.2.2 パス ... 15
5.2.3 真偽値と null ... 16
5.2.4 リスト ... 16
5.2.5 アトリビュートセット（Attribute Set） ... 16
5.3 関数 ... 17
5.3.1 アトリビュートセットを引数に取る関数 ... 17
5.4 let 式と with 式 ... 17
5.5 if 式 ... 18
5.6 inherit キーワード ... 18
5.7 import ... 18
5.8 builtins ... 18
6 開発環境構築 ... 20
6.1 nix-shell による開発環境 ... 20
6.1.1 基本的な使い方 ... 20
6.1.2 shell.nix による環境定義 ... 20
6.2 nix develop による開発環境（Flakes 対応） ... 21
6.3 言語別の開発環境例 ... 22
6.3.1 Rust 開発環境 ... 22
6.3.2 Go 開発環境 ... 22
6.3.3 Node.js 開発環境 ... 22
6.4 direnv との連携 ... 23
6.4.1 セットアップ ... 23
6.4.2 プロジェクトでの使用 ... 23
7 Nix ファイルの書き方 ... 24
7.1 パッケージの定義（default.nix） ... 24
7.1.1 基本的な構造 ... 24
7.1.2 主要な属性 ... 24
7.2 ソースの取得 ... 25
7.2.1 GitHub からの取得 ... 25
7.2.2 URL からの取得 ... 25
7.3 言語固有のビルダー ... 25
7.3.1 Python パッケージ ... 25
7.3.2 Rust パッケージ ... 26
7.3.3 Go パッケージ ... 26
7.4 オーバーレイ（Overlay） ... 27
7.5 override と overrideAttrs ... 27
8 Flakes ... 28
8.1 Flakes とは ... 28
8.2 Flakes の有効化 ... 28
8.3 flake.nix の構造 ... 28
8.3.1 inputs（入力） ... 28
8.3.2 outputs（出力） ... 29

8.4 flake.lock 29
8.5 Flake の操作 30
8.6 複数システム対応 30
8.7 実践例: Web アプリケーション 31
9 NixOS 基礎 33
9.1 NixOS とは 33
9.2 configuration.nix の基本構造 33
9.3 システムの再構築 34
9.4 サービス管理 34
9.5 世代管理とロールバック 35
9.6 NixOS モジュールシステム 35
9.7 Flakes で NixOS を管理 36
10 Tips とベストプラクティス 38
10.1 ディスク容量の管理 38
10.2 よく使うコマンド一覧 38
10.3 デバッグ手法 39
10.3.1 ビルドエラーの調査 39
10.3.2 Nix 式の評価 39
10.3.3 依存関係の調査 39
10.4 セキュリティ 39
10.4.1 信頼されたユーザーの設定 39
10.4.2 バイナリキャッシュの署名検証 40
10.5 プロジェクト構成のベストプラクティス 40
10.5.1 .gitignore に追加すべきもの 40
10.6 コミュニティリソース 40

1 はじめに

1.1 Nix とは

Nix は、**再現性**と**宣言的な構成管理**を中核とするパッケージマネージャおよびビルドシステムである。2003 年に Eelco Dolstra の博士論文から生まれた Nix は、従来のパッケージ管理の根本的な問題——「自分の環境では動くのに」——を解決するために設計された。

Nix の主な特徴は以下の通りである。

- **純粋関数型パッケージ管理**: パッケージを副作用のない関数として扱い、入力が同じなら常に同じ結果を得る
- **再現可能なビルド**: 同じ Nix 式からは、いつ・どこでビルドしても同一の成果物が生成される
- **アトミックなアップグレードとロールバック**: システム変更を安全に適用・取り消しできる
- **複数バージョンの共存**: 同一パッケージの異なるバージョンを競合なく同時にインストールできる
- **宣言的なシステム構成**: システム全体の状態を設定ファイルとして記述できる

1.2 Nix エコシステムの全体像

Nix エコシステムは複数のコンポーネントから構成されている。

コンポーネント	説明
Nix	パッケージマネージャ本体。パッケージのビルド・インストール・管理を行う
Nix 言語	パッケージやシステム構成を記述するための純粋関数型言語
Nixpkgs	80,000 以上のパッケージを収録する世界最大級のパッケージリポジトリ
NixOS	Nix 上に構築された Linux ディストリビューション。OS 全体を宣言的に管理できる
Flakes	プロジェクトの依存関係と出力を標準化する新しい仕組み

1.3 他のパッケージマネージャとの比較

特徴	Nix	apt/dnf	Homebrew
再現性	完全な再現性	バージョン固定が困難	部分的
ロールバック	アトミック	困難	非対応
複数バージョン共存	標準対応	非対応	限定的
クロスプラットフォーム	Linux / macOS	Linux のみ	macOS / Linux

特徴	Nix	apt/dnf	Homebrew
宣言的構成	対応	非対応	非対応（Bundle で一部対応）
パッケージ数	80,000+	60,000+	6,000+

1.4 本書の対象読者

本書は以下のような読者を想定している。

- Linux や macOS でのコマンドライン操作に慣れている方
- パッケージ管理やシステム構成に興味がある方
- 開発環境の再現性に課題を感じている方
- NixOS への移行を検討している方

1.5 前提知識

- 基本的なターミナル操作（`cd`、`ls`、`mkdir` など）
- テキストエディタの使用経験
- パッケージマネージャ（apt、yum、brew など）の基本的な理解

2 インストール

2.1 システム要件

Nix は以下の環境で動作する。

項目	要件
OS	Linux（x86_64, aarch64）、macOS（x86_64, aarch64）
ディスク容量	最低 1GB（Nix Store 用に 10GB 以上推奨）
メモリ	最低 512MB（ビルド時は 2GB 以上推奨）
権限	root 権限または sudo 権限（マルチユーザーインストール時）

2.2 インストール方法

Nix には 2 つのインストールモードがある。

項目	シングルユーザー	マルチユーザー（推奨）
Nix Store の所有者	現在のユーザー	root
ビルドの分離	なし	専用ビルドユーザーで分離
セキュリティ	低	高
セットアップの容易さ	簡単	やや複雑（自動化されている）
推奨環境	テスト・試用	本番・チーム開発

2.2.1 マルチユーザーインストール（推奨）

公式のインストールスクリプトを使用する。

```
sh <(curl -L https://nixos.org/nix/install) --daemon
```

インストールが完了したら、シェルを再起動するか以下を実行する。

```
. /nix/var/nix/profiles/default/etc/profile.d/nix-daemon.sh
```

2.2.2 シングルユーザーインストール

```
sh <(curl -L https://nixos.org/nix/install) --no-daemon
```

シェルを再起動するか以下を実行する。

```
. ~/.nix-profile/etc/profile.d/nix.sh
```

2.3 インストールの確認

インストールが正しく完了したか確認する。

```
nix --version
```

以下のように表示されれば成功である。

```
nix (Nix) 2.24.x
```

簡単なテストとして、`hello` パッケージを実行してみる。

```
nix-shell -p hello --run hello
```

`Hello, world!` と表示されれば、Nix は正常に動作している。

2.4 Nix の実験的機能を有効にする

Flakes や新しい `nix` コマンド体系を使用するには、実験的機能を有効にする必要がある。`~/.config/nix/nix.conf` を作成または編集する。

```
mkdir -p ~/.config/nix
echo "experimental-features = nix-command flakes" >> ~/.config/nix/nix.conf
```

これにより `nix build`、`nix run`、`nix develop` などの新しいコマンドと Flakes が利用可能になる。

2.5 アンインストール

2.5.1 Linux（マルチユーザー）

```
sudo systemctl stop nix-daemon.service
sudo systemctl disable nix-daemon.socket nix-daemon.service
sudo rm -rf /nix /etc/nix ~/.nix-profile ~/.nix-defexpr ~/.nix-channels
```

さらに、`/etc/bashrc` や `/etc/zshrc` から Nix 関連の行を削除する。

2.5.2 macOS

```
sudo rm -rf /nix
sudo rm -rf /etc/nix
rm -rf ~/.nix-profile ~/.nix-defexpr ~/.nix-channels
```

`/etc/bashrc`、`/etc/zshrc`、`/etc/synthetic.conf`、`/etc/fstab` から Nix 関連の設定を削除する。

3 基本概念

3.1 Nix Store

Nix Store は、Nix が管理するすべてのパッケージやビルド成果物を格納する中央リポジトリである。デフォルトでは `/nix/store` に配置される。

各パッケージは以下のようなパスに格納される。

```
/nix/store/<ハッシュ>-<パッケージ名>-<バージョン>/
```

例えば、GNU Hello パッケージは以下のようになる。

```
/nix/store/qy93dp4a3rqyn2mz63fbxjg228hffwyw-hello-2.12.1/
```

ハッシュはパッケージのビルドに使用されたすべての入力（ソースコード、依存関係、ビルドスクリプト、コンパイラ）から計算される。入力が 1 ビットでも異なれば、異なるハッシュが生成される。

3.1.1 Nix Store の特性

- **不変（Immutable）**: Store 内のパスは作成後に変更されない
- **コンテンツアドレス**: パスはその内容のハッシュに基づく
- **共有**: 同一の成果物は一度だけ格納される（重複排除）
- **ガベージコレクション**: 不要になったパスは `nix-collect-garbage` で回収できる

3.2 デリベーション（Derivation）

デリベーションは Nix におけるビルドの基本単位である。「何をどうビルドするか」を記述した設計図のようなものである。

デリベーションには以下の情報が含まれる。

- **ビルドに必要な入力**（ソースコード、依存パッケージ）
- **ビルドスクリプト**（ビルドの手順）
- **出力先**（Nix Store 内のパス）
- **ビルド環境の設定**（環境変数など）

```nix
# 概念的な例
derivation {
  name = "my-package";
  builder = "${bash}/bin/bash";
  src = ./src;
  buildInputs = [ gcc make ];
}
```

デリベーションの重要な性質は**純粋性**である。同じ入力からは常に同じ出力が得られるよう、ビルドはネットワークアクセスのないサンドボックス環境で実行される。

3.3 プロファイルと世代

Nix は**プロファイル**を使って、ユーザーの環境（インストール済みパッケージの集合）を管理する。

```
~/.nix-profile -> /nix/var/nix/profiles/per-user/<user>/profile
```

パッケージをインストールまたは削除するたびに、新しい**世代（Generation）**が作成される。

```
# 世代の一覧を確認
nix-env --list-generations

# 出力例:
#   1   2024-01-15 10:00:00
#   2   2024-01-16 14:30:00   (current)
```

世代管理により、いつでも以前の状態に戻すことができる。

```
# 1つ前の世代に戻す
nix-env --rollback

# 特定の世代に切り替え
nix-env --switch-generation 1
```

3.4 チャネル

チャネルは Nixpkgs リポジトリの特定のスナップショットを指す名前付き参照である。

```
# 現在のチャネルを確認
nix-channel --list

# チャネルの追加
nix-channel --add https://nixos.org/channels/nixpkgs-unstable nixpkgs

# チャネルの更新
nix-channel --update
```

主なチャネルは以下の通りである。

チャネル	説明
nixpkgs-unstable	最新のパッケージ。テスト済みだが安定性は保証されない
nixos-24.11	NixOS 24.11 のリリースチャネル。安定版
nixos-unstable	NixOS 向けの最新チャネル

Flakes を使用する場合、チャネルの代わりに `flake.lock` で依存関係のバージョンを固定する。

3.5 クロージャ（Closure）

パッケージのクロージャとは、そのパッケージが動作するために必要なすべての依存関係を含む完全な集合である。

```
# パッケージのクロージャを確認
nix-store -qR $(which hello)
```

この出力には、`hello` の実行に必要なすべてのライブラリやランタイム依存が含まれる。クロージャの概念により、Nix はパッケージの完全な可搬性を実現している。

4 パッケージ管理

4.1 パッケージの検索

4.1.1 コマンドラインでの検索

```
# 旧コマンド
nix-env -qaP 'firefox'

# 新コマンド（experimental-features 有効時）
nix search nixpkgs firefox
```

`nix search` コマンドはパッケージ名と説明文の両方を検索対象とする。

```
# 正規表現による検索
nix search nixpkgs 'python3.*Packages\.numpy'
```

4.1.2 Web での検索

Nixpkgs のパッケージは以下の Web サイトからも検索できる。

- **search.nixos.org**: 公式のパッケージ検索サイト

4.2 パッケージのインストール

4.2.1 命令的なインストール（nix-env）

`nix-env` はユーザープロファイルにパッケージをインストールする従来のコマンドである。

```
# パッケージのインストール
nix-env -iA nixpkgs.firefox

# 複数パッケージの一括インストール
nix-env -iA nixpkgs.git nixpkgs.vim nixpkgs.tmux
```

4.2.2 一時的な利用（nix-shell / nix shell）

パッケージをプロファイルにインストールせず、一時的に使用することもできる。

```
# 旧コマンド: パッケージを含む一時シェルを起動
nix-shell -p python3 nodejs

# 新コマンド: パッケージを含む一時シェルを起動
nix shell nixpkgs#python3 nixpkgs#nodejs
```

```
# コマンドを1回だけ実行
nix run nixpkgs#cowsay -- "Hello, Nix!"
```

この方法はシステムを汚さずにツールを試したい場合に便利である。

4.3 パッケージの削除

```
# パッケージの削除
nix-env -e firefox

# 不要なパッケージのガベージコレクション
nix-collect-garbage

# 古い世代も含めてクリーンアップ（30日以上前の世代を削除）
nix-collect-garbage --delete-older-than 30d
```

4.4 インストール済みパッケージの管理

```
# インストール済みパッケージの一覧
nix-env -q

# 詳細情報付きで一覧
nix-env -q --description

# パッケージのアップグレード
nix-env -u            # すべてのパッケージをアップグレード
nix-env -uA nixpkgs.firefox  # 特定のパッケージをアップグレード
```

4.5 宣言的なパッケージ管理

命令的な `nix-env` の代わりに、設定ファイルで管理する宣言的な方法が推奨される。

4.5.1 home-manager による管理

home-manager を使うと、ユーザー環境を宣言的に管理できる。

```
# ~/.config/home-manager/home.nix
{ config, pkgs, ... }:

{
  home.username = "myuser";
  home.homeDirectory = "/home/myuser";
  home.stateVersion = "24.11";
```

```
  home.packages = with pkgs; [
    firefox
    git
    vim
    tmux
    ripgrep
    fd
    jq
  ];

  programs.home-manager.enable = true;
}
```

```
# 設定を適用
home-manager switch
```

この方法であれば、環境の構成を Git で管理し、別のマシンでも同一の環境を再現できる。

4.6 バイナリキャッシュ

Nix はデフォルトで `cache.nixos.org` のバイナリキャッシュを使用する。ビルド済みの成果物がキャッシュに存在すれば、ソースからビルドする代わりにダウンロードされる。

```
# キャッシュの設定確認
nix show-config | grep substituters

# カスタムキャッシュの追加（nix.conf）
# substituters = https://cache.nixos.org https://my-cache.example.com
# trusted-public-keys = cache.nixos.org-1:... my-cache-1:...
```

バイナリキャッシュにより、大半のパッケージはダウンロードするだけで利用でき、ビルド時間を大幅に短縮できる。

5 Nix 言語基礎

5.1 Nix 言語の概要

Nix 言語は、パッケージやシステム構成を記述するために設計された**純粋関数型**の**遅延評価**言語である。汎用プログラミング言語ではなく、ビルドの記述に特化したドメイン固有言語（DSL）である。

Nix 言語の特徴は以下の通りである。

- **純粋**: 同じ入力に対して常に同じ結果を返す
- **遅延評価**: 値は実際に必要になるまで評価されない
- **動的型付け**: 型は実行時に検査される

`nix repl` コマンドで対話的に Nix 言語を試すことができる。

```
nix repl
```

5.2 基本的なデータ型

5.2.1 数値と文字列

```
# 整数
42

# 文字列（ダブルクォート）
"hello, world"

# 複数行文字列（2つのシングルクォート）
''
  This is a
  multi-line string.
''

# 文字列補間
let name = "Nix"; in "Hello, ${name}!"
# => "Hello, Nix!"
```

5.2.2 パス

Nix 言語にはパスのリテラルがある。

```
# 相対パス
./src/main.c

# 絶対パス
```

```
/etc/nixos/configuration.nix

# ホームディレクトリ
~/.config/nix/nix.conf
```

パスは文字列とは異なるデータ型であり、ファイルシステムの参照として扱われる。

5.2.3 真偽値と null

```
true
false
null
```

5.2.4 リスト

```
# リスト（要素はスペースで区切る、カンマは不要）
[ 1 2 3 ]

# 異なる型の混在が可能
[ 1 "hello" true ./path ]

# リストの結合
[ 1 2 ] ++ [ 3 4 ]
# => [ 1 2 3 4 ]
```

5.2.5 アトリビュートセット（Attribute Set）

アトリビュートセットはNix言語で最も重要なデータ構造である。他の言語における辞書やマップに相当する。

```
# 基本的なアトリビュートセット
{
  name = "my-package";
  version = "1.0.0";
  src = ./src;
}

# ネストしたアクセス
let config = {
  server = {
    host = "localhost";
    port = 8080;
  };
};
in config.server.port
# => 8080
```

```
# 再帰的アトリビュートセット（rec）
rec {
  x = 1;
  y = x + 1;
}
# => { x = 1; y = 2; }
```

5.3 関数

Nix 言語の関数はすべて無名関数（ラムダ）であり、引数を 1 つだけ取る。

```
# 基本的な関数
x: x + 1

# 関数の適用
let increment = x: x + 1;
in increment 5
# => 6

# 複数引数（カリー化）
a: b: a + b

let add = a: b: a + b;
in add 3 4
# => 7
```

5.3.1 アトリビュートセットを引数に取る関数

Nix で最も一般的なパターンである。

```
# 基本形
{ name, version }: "Package: ${name}-${version}"

# デフォルト値
{ name, version ? "0.1.0" }: "Package: ${name}-${version}"

# 残りの引数を許容（...）
{ name, version, ... }: "Package: ${name}-${version}"
```

5.4 let 式と with 式

```
# let式: ローカル束縛
let
  x = 1;
  y = 2;
```

```
in x + y
# => 3

# with式: アトリビュートセットのスコープ導入
let pkgs = { git = "git-2.43"; vim = "vim-9.1"; };
in with pkgs; [ git vim ]
# => [ "git-2.43" "vim-9.1" ]
```

5.5 if 式

```
if 3 > 2 then "yes" else "no"
# => "yes"
```

Nix 言語では `if` は式であり、常に値を返す。`else` は省略できない。

5.6 inherit キーワード

`inherit` は変数をアトリビュートセットに簡潔に取り込むための構文糖衣である。

```
let
  name = "hello";
  version = "1.0";
in {
  inherit name version;
  # 以下と同等:
  # name = name;
  # version = version;
}
```

5.7 import

他の Nix ファイルを読み込むには `import` を使用する。

```
# ファイルを読み込み・評価
import ./config.nix

# 引数付きで読み込み
import ./package.nix { inherit pkgs; }
```

5.8 builtins

Nix には多数の組み込み関数（builtins）が用意されている。

```
# 代表的な builtins
builtins.length [ 1 2 3 ]          # => 3
builtins.map (x: x * 2) [ 1 2 3 ]  # => [ 2 4 6 ]
builtins.filter (x: x > 2) [ 1 2 3 4 ]  # => [ 3 4 ]
builtins.attrNames { a = 1; b = 2; }    # => [ "a" "b" ]
builtins.hasAttr "a" { a = 1; }          # => true
builtins.readFile ./README.md
builtins.toJSON { name = "test"; }
```

`nix repl` で `builtins.` と入力してタブ補完すると、利用可能な組み込み関数の一覧を確認できる。

6 開発環境構築

6.1 nix-shell による開発環境

`nix-shell` はプロジェクト固有の開発環境を一時的に構築するコマンドである。必要なツールやライブラリを含むシェルを起動し、作業終了後にはクリーンに破棄できる。

6.1.1 基本的な使い方

```
# パッケージを指定して一時シェルを起動
nix-shell -p python3 python3Packages.requests nodejs

# シェル内でツールが利用可能
python3 --version
node --version

# シェルから抜ける
exit
```

6.1.2 shell.nix による環境定義

プロジェクトルートに `shell.nix` を配置することで、チーム全員が同一の開発環境を利用できる。

```
# shell.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.mkShell {
  buildInputs = with pkgs; [
    python3
    python3Packages.requests
    python3Packages.flask
    nodejs
    nodePackages.npm
    postgresql
    redis
  ];

  shellHook = ''
    echo "開発環境が準備されました"
    export DATABASE_URL="postgresql://localhost/myapp"
    export FLASK_ENV="development"
  '';
}
```

```
# shell.nix があるディレクトリで実行
nix-shell
```

`shellHook` にはシェル起動時に実行するコマンドを記述できる。環境変数の設定やサービスの起動などに活用する。

6.2 nix develop による開発環境（Flakes 対応）

Flakes を使用する場合は、`nix develop` コマンドで開発環境を構築する。

```
# flake.nix
{
  description = "My Python project";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-24.11";
  };

  outputs = { self, nixpkgs }:
    let
      system = "x86_64-linux";
      pkgs = nixpkgs.legacyPackages.${system};
    in {
      devShells.${system}.default = pkgs.mkShell {
        buildInputs = with pkgs; [
          python3
          python3Packages.requests
          python3Packages.flask
          nodePackages.pyright
        ];

        shellHook = ''
          echo "Python開発環境が準備されました"
        '';
      };
    };
}
```

```
# 開発環境に入る
nix develop

# 特定の devShell を指定
nix develop .#myShell
```

6.3 言語別の開発環境例

6.3.1 Rust 開発環境

```
# shell.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.mkShell {
  buildInputs = with pkgs; [
    rustc
    cargo
    rustfmt
    clippy
    rust-analyzer
    pkg-config
    openssl
  ];

  RUST_SRC_PATH = "${pkgs.rust.packages.stable.rustPlatform.rustLibSrc}";
}
```

6.3.2 Go 開発環境

```
# shell.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.mkShell {
  buildInputs = with pkgs; [
    go
    gopls
    gotools
    go-tools
    delve
  ];

  shellHook = ''
    export GOPATH=$HOME/go
    export PATH=$GOPATH/bin:$PATH
  '';
}
```

6.3.3 Node.js 開発環境

```
# shell.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.mkShell {
```

```
  buildInputs = with pkgs; [
    nodejs_20
    nodePackages.npm
    nodePackages.typescript
    nodePackages.typescript-language-server
  ];
}
```

6.4 direnv との連携

`direnv` と `nix-direnv` を組み合わせることで、ディレクトリに入るだけで自動的に Nix 開発環境を有効化できる。

6.4.1 セットアップ

```
# home.nix（home-manager を使用する場合）
{
  programs.direnv = {
    enable = true;
    nix-direnv.enable = true;
  };
}
```

6.4.2 プロジェクトでの使用

プロジェクトルートに `.envrc` ファイルを作成する。

```
# .envrc（shell.nix を使用する場合）
use nix

# .envrc（flake.nix を使用する場合）
use flake
```

```
# direnv を許可
direnv allow
```

これにより、プロジェクトディレクトリに `cd` するだけで自動的に開発環境が有効化され、離れると無効化される。エディタや IDE との連携も自動で行われるため、非常に快適な開発体験が得られる。

7 Nix ファイルの書き方

7.1 パッケージの定義（default.nix）

Nix でパッケージを定義する基本的な方法は、`mkDerivation` 関数を使用することである。

7.1.1 基本的な構造

```
# default.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.stdenv.mkDerivation {
  pname = "my-app";
  version = "1.0.0";

  src = ./.;

  buildInputs = [ pkgs.zlib pkgs.openssl ];
  nativeBuildInputs = [ pkgs.cmake pkgs.pkg-config ];

  buildPhase = ''
    cmake .
    make
  '';

  installPhase = ''
    mkdir -p $out/bin
    cp my-app $out/bin/
  '';
}
```

7.1.2 主要な属性

属性	説明
`pname`	パッケージ名
`version`	バージョン
`src`	ソースコード（パス、fetchurl、fetchFromGitHub など）
`buildInputs`	実行時に必要な依存パッケージ
`nativeBuildInputs`	ビルド時のみ必要なツール
`buildPhase`	ビルド手順
`installPhase`	インストール手順（`$out` がインストール先）
`checkPhase`	テスト手順
`meta`	パッケージのメタ情報（説明、ライセンスなど）

7.2 ソースの取得

7.2.1 GitHub からの取得

```
{ pkgs ? import <nixpkgs> {} }:

pkgs.stdenv.mkDerivation {
  pname = "example";
  version = "1.2.3";

  src = pkgs.fetchFromGitHub {
    owner = "example";
    repo = "example-repo";
    rev = "v1.2.3";
    sha256 = "sha256-AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=";
  };

  # ...
}
```

ハッシュが不明な場合は、仮の値を入れてビルドを実行すると、正しいハッシュがエラーメッセージに表示される。

```
# ハッシュを事前に計算
nix-prefetch-url --unpack https://github.com/example/repo/archive/v1.2.3.tar.gz
```

7.2.2 URL からの取得

```
src = pkgs.fetchurl {
  url = "https://example.com/source-1.0.tar.gz";
  sha256 = "sha256-...";
};
```

7.3 言語固有のビルダー

7.3.1 Python パッケージ

```
{ pkgs ? import <nixpkgs> {} }:

pkgs.python3Packages.buildPythonApplication {
  pname = "my-python-app";
  version = "1.0.0";
  src = ./.;

  propagatedBuildInputs = with pkgs.python3Packages; [
```

```
    requests
    flask
    sqlalchemy
  ];

  checkInputs = with pkgs.python3Packages; [
    pytest
  ];

  checkPhase = ''
    pytest tests/
  '';
}
```

7.3.2 Rust パッケージ

```
{ pkgs ? import <nixpkgs> {} }:

pkgs.rustPlatform.buildRustPackage {
  pname = "my-rust-app";
  version = "1.0.0";
  src = ./.;

  cargoLock = {
    lockFile = ./Cargo.lock;
  };

  nativeBuildInputs = [ pkgs.pkg-config ];
  buildInputs = [ pkgs.openssl ];
}
```

7.3.3 Go パッケージ

```
{ pkgs ? import <nixpkgs> {} }:

pkgs.buildGoModule {
  pname = "my-go-app";
  version = "1.0.0";
  src = ./.;

  vendorHash = "sha256-...";

  # テストをスキップする場合
  doCheck = false;
}
```

7.4 オーバーレイ（Overlay）

オーバーレイを使うと、既存の Nixpkgs パッケージを変更したり、新しいパッケージを追加したりできる。

```
# overlay.nix
final: prev: {
  # 既存パッケージのオーバーライド
  vim = prev.vim.override {
    pythonSupport = true;
  };

  # 新しいパッケージの追加
  my-tool = final.callPackage ./my-tool.nix {};
}
```

```
# オーバーレイの適用
let
  pkgs = import <nixpkgs> {
    overlays = [ (import ./overlay.nix) ];
  };
in pkgs.vim  # Python サポートが有効化された vim
```

7.5 override と overrideAttrs

既存パッケージの一部を変更するには `override` や `overrideAttrs` を使用する。

```
# override: 関数引数の変更
pkgs.ffmpeg.override {
  withVpx = true;
  withX264 = true;
}

# overrideAttrs: デリベーション属性の変更
pkgs.hello.overrideAttrs (oldAttrs: {
  version = "2.13";
  src = pkgs.fetchurl {
    url = "https://ftp.gnu.org/gnu/hello/hello-2.13.tar.gz";
    sha256 = "sha256-...";
  };
  patches = [ ./my-fix.patch ];
})
```

`override` は高水準の設定変更に、`overrideAttrs` は低水準のビルド属性の変更に適している。

8 Flakes

8.1 Flakes とは

Flakes は Nix プロジェクトの依存関係と出力を標準化する仕組みである。従来の Nix では、チャネルや NIX_PATH など環境に依存する要素が再現性を損なう原因となっていた。Flakes はこの問題を解決し、完全に再現可能な Nix 式を実現する。

Flakes の主な特徴は以下の通りである。

- **ロックファイル**（`flake.lock`）による依存関係の完全な固定
- **標準化された構造**による統一的なプロジェクト管理
- **純粋な評価**: 環境変数に依存しない
- **構成可能性**: Flakes 同士を容易に組み合わせられる

8.2 Flakes の有効化

Flakes は現在も実験的機能であるため、明示的に有効化する必要がある。

```
# ~/.config/nix/nix.conf
experimental-features = nix-command flakes
```

8.3 flake.nix の構造

`flake.nix` は Flake のエントリーポイントであり、以下の 3 つの主要な属性を持つ。

```
{
  description = "プロジェクトの説明";

  inputs = {
    # 依存する他の Flake
  };

  outputs = { self, ... }@inputs: {
    # このFlakeが提供する成果物
  };
}
```

8.3.1 inputs（入力）

```
inputs = {
  # Nixpkgs（最も一般的な入力）
  nixpkgs.url = "github:NixOS/nixpkgs/nixos-24.11";
```

```
  # 別のFlake
  home-manager = {
    url = "github:nix-community/home-manager/release-24.11";
    inputs.nixpkgs.follows = "nixpkgs";  # nixpkgsを共有
  };

  # Git リポジトリ
  my-lib.url = "git+https://example.com/my-lib.git";

  # ローカルパス
  my-utils.url = "path:./utils";
};
```

`follows` を使うと、間接的な依存関係のバージョンを統一できる。これによりビルド時間の短縮とストアの節約ができる。

8.3.2 outputs（出力）

```
outputs = { self, nixpkgs, ... }:
  let
    system = "x86_64-linux";
    pkgs = nixpkgs.legacyPackages.${system};
  in {
    # パッケージ
    packages.${system}.default = pkgs.callPackage ./package.nix {};
    packages.${system}.my-tool = pkgs.callPackage ./my-tool.nix {};

    # 開発シェル
    devShells.${system}.default = pkgs.mkShell {
      buildInputs = [ pkgs.go pkgs.gopls ];
    };

    # NixOS モジュール
    nixosModules.default = import ./module.nix;

    # オーバーレイ
    overlays.default = final: prev: {
      my-tool = final.callPackage ./my-tool.nix {};
    };
  };
```

8.4 flake.lock

`flake.lock` はすべての入力の正確なリビジョンを記録するファイルである。Git にコミットして共有することで、全員が同一のバージョンを使用できる。

```
# ロックファイルの更新（すべての入力）
nix flake update

# 特定の入力のみ更新
nix flake update nixpkgs

# ロックファイルの情報を表示
nix flake metadata
```

8.5 Flake の操作

```
# Flakeの初期化
nix flake init

# テンプレートから初期化
nix flake init -t templates#rust

# Flakeの情報を表示
nix flake show
nix flake metadata

# パッケージのビルド
nix build .#my-tool

# パッケージの実行
nix run .#my-tool

# 開発環境に入る
nix develop

# Flake の入力を確認
nix flake metadata
```

8.6 複数システム対応

複数のアーキテクチャに対応するには、`flake-utils` を使用すると便利である。

```
{
  description = "Cross-platform project";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-24.11";
    flake-utils.url = "github:numtide/flake-utils";
  };

  outputs = { self, nixpkgs, flake-utils }:
```

```
    flake-utils.lib.eachDefaultSystem (system:
      let pkgs = nixpkgs.legacyPackages.${system};
      in {
        packages.default = pkgs.callPackage ./package.nix {};

        devShells.default = pkgs.mkShell {
          buildInputs = with pkgs; [ go gopls ];
        };
      }
    );
}
```

`eachDefaultSystem` は `x86_64-linux`、`aarch64-linux`、`x86_64-darwin`、`aarch64-darwin` の 4 つのシステムに対して自動的に出力を生成する。

8.7 実践例: Web アプリケーション

```
{
  description = "My Web Application";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-24.11";
    flake-utils.url = "github:numtide/flake-utils";
  };

  outputs = { self, nixpkgs, flake-utils }:
    flake-utils.lib.eachDefaultSystem (system:
      let pkgs = nixpkgs.legacyPackages.${system};
      in {
        packages.default = pkgs.buildGoModule {
          pname = "my-webapp";
          version = "0.1.0";
          src = ./.;
          vendorHash = "sha256-...";
        };

        devShells.default = pkgs.mkShell {
          buildInputs = with pkgs; [
            go
            gopls
            nodejs
            postgresql
            redis
          ];

          shellHook = ''
            echo "Web App 開発環境が準備されました"
            export DATABASE_URL="postgresql://localhost/myapp"
```

```
            '';
          };
        }
      );
}
```

9 NixOS 基礎

9.1 NixOS とは

NixOS は Nix パッケージマネージャを基盤とした Linux ディストリビューションである。OS 全体の構成——カーネル、サービス、ユーザー、ネットワーク——をすべて 1 つの設定ファイルで宣言的に管理できる。

NixOS の主な特徴は以下の通りである。

- **宣言的なシステム構成**: `/etc/nixos/configuration.nix` でシステム全体を定義
- **アトミックなアップグレードとロールバック**: 失敗したアップグレードからブートローダーで復旧可能
- **再現性**: 同じ設定ファイルから同じシステムを構築できる
- **テスト容易性**: 仮想マシンでシステム構成をテストできる

9.2 configuration.nix の基本構造

```
# /etc/nixos/configuration.nix
{ config, pkgs, ... }:

{
  # ブートローダー
  boot.loader.systemd-boot.enable = true;
  boot.loader.efi.canTouchEfiVariables = true;

  # ネットワーク
  networking.hostName = "my-nixos";
  networking.networkmanager.enable = true;

  # タイムゾーン
  time.timeZone = "Asia/Tokyo";

  # ロケール
  i18n.defaultLocale = "ja_JP.UTF-8";

  # システムパッケージ
  environment.systemPackages = with pkgs; [
    vim
    git
    firefox
    wget
    curl
  ];

  # SSH サーバー
```

```
  services.openssh.enable = true;

  # ファイアウォール
  networking.firewall = {
    enable = true;
    allowedTCPPorts = [ 22 80 443 ];
  };

  # ユーザー
  users.users.myuser = {
    isNormalUser = true;
    extraGroups = [ "wheel" "networkmanager" ];
    shell = pkgs.zsh;
  };

  # NixOS のバージョン（変更しないこと）
  system.stateVersion = "24.11";
}
```

9.3 システムの再構築

設定を変更したら、以下のコマンドで適用する。

```
# 設定を適用してアクティブ化
sudo nixos-rebuild switch

# テスト（再起動後は元に戻る）
sudo nixos-rebuild test

# ビルドのみ（適用しない）
sudo nixos-rebuild build

# ブートエントリに追加するが、現在のセッションには適用しない
sudo nixos-rebuild boot
```

9.4 サービス管理

NixOS では systemd サービスを宣言的に管理する。

```
# Nginx の有効化
services.nginx = {
  enable = true;
  virtualHosts."example.com" = {
    root = "/var/www/example.com";
    locations."/" = {
      tryFiles = "$uri $uri/ =404";
```

```
    };
  };
};

# PostgreSQL の有効化
services.postgresql = {
  enable = true;
  package = pkgs.postgresql_16;
  authentication = ''
    local all all trust
    host all all 127.0.0.1/32 trust
  '';
};

# Docker の有効化
virtualisation.docker.enable = true;
users.users.myuser.extraGroups = [ "docker" ];
```

NixOS のオプション検索サイト（search.nixos.org/options）で利用可能な全オプションを確認できる。

9.5 世代管理とロールバック

NixOS は `nixos-rebuild switch` のたびに新しいシステム世代を作成する。

```
# システム世代の一覧
sudo nix-env -p /nix/var/nix/profiles/system --list-generations

# 1つ前の世代にロールバック
sudo nixos-rebuild switch --rollback

# 特定の世代に切り替え
sudo nix-env -p /nix/var/nix/profiles/system --switch-generation 42
sudo /nix/var/nix/profiles/system/bin/switch-to-configuration switch
```

ブートローダーにも各世代のエントリが表示されるため、起動時にも以前のシステム状態を選択できる。

9.6 NixOS モジュールシステム

NixOS の設定はモジュールシステムで構成されている。設定ファイルを分割して管理できる。

```
# /etc/nixos/configuration.nix
{ config, pkgs, ... }:

{
  imports = [
```

```
    ./hardware-configuration.nix  # ハードウェア設定（自動生成）
    ./networking.nix              # ネットワーク設定
    ./services.nix                # サービス設定
    ./users.nix                   # ユーザー設定
  ];

  # 共通設定
  system.stateVersion = "24.11";
}
```

```
# /etc/nixos/services.nix
{ config, pkgs, ... }:

{
  services.openssh.enable = true;
  services.nginx.enable = true;
}
```

9.7 Flakes で NixOS を管理

Flakes で NixOS の構成を管理することで、再現性をさらに向上させることができる。

```
# flake.nix
{
  description = "My NixOS Configuration";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-24.11";
    home-manager = {
      url = "github:nix-community/home-manager/release-24.11";
      inputs.nixpkgs.follows = "nixpkgs";
    };
  };

  outputs = { self, nixpkgs, home-manager, ... }: {
    nixosConfigurations.my-nixos = nixpkgs.lib.nixosSystem {
      system = "x86_64-linux";
      modules = [
        ./configuration.nix
        home-manager.nixosModules.home-manager
        {
          home-manager.useGlobalPkgs = true;
          home-manager.useUserPackages = true;
          home-manager.users.myuser = import ./home.nix;
        }
      ];
    };
```

```
  };
}
```

```
# Flakes を使ったシステム再構築
sudo nixos-rebuild switch --flake .#my-nixos
```

10 Tips とベストプラクティス

10.1 ディスク容量の管理

Nix Store は使い続けるとディスク容量を大量に消費する。定期的なクリーンアップが重要である。

```
# ガベージコレクション（未参照のパスを削除）
nix-collect-garbage

# 古い世代を削除してからガベージコレクション
nix-collect-garbage --delete-older-than 14d

# Nix Store の最適化（ハードリンクによる重複排除）
nix store optimise

# 現在の Nix Store のサイズを確認
du -sh /nix/store
```

自動的にガベージコレクションを実行するには、NixOS で以下の設定を追加する。

```
# configuration.nix
nix.gc = {
  automatic = true;
  dates = "weekly";
  options = "--delete-older-than 30d";
};
```

10.2 よく使うコマンド一覧

旧コマンド	新コマンド（Flakes 対応）
`nix-build`	`nix build`
`nix-shell -p pkg`	`nix shell nixpkgs#pkg`
`nix-shell`	`nix develop`
`nix-env -iA nixpkgs.pkg`	`nix profile install nixpkgs#pkg`
`nix-env -e pkg`	`nix profile remove pkg`
`nix-env -q`	`nix profile list`
`nix-channel --update`	`nix flake update`
`nix-env -qaP pkg`	`nix search nixpkgs pkg`

10.3 デバッグ手法

10.3.1 ビルドエラーの調査

```
# 詳細なビルドログを表示
nix build .#my-package -L

# ビルドログを保持
nix build .#my-package --keep-failed

# 失敗したビルドのシェルに入る
nix develop .#my-package
# 手動でビルドステップを再現して問題を特定
```

10.3.2 Nix 式の評価

```
# Nix式を評価して結果を表示
nix eval .#packages.x86_64-linux.default.name

# nix repl で対話的にデバッグ
nix repl
# nix-repl> :l <nixpkgs>
# nix-repl> python3.version
# => "3.12.x"
```

10.3.3 依存関係の調査

```
# パッケージの依存ツリーを表示
nix-store -q --tree $(nix build .#my-package --print-out-paths)

# パッケージがなぜ依存されているか調査
nix why-depends .#my-package .#openssl
```

10.4 セキュリティ

10.4.1 信頼されたユーザーの設定

```
# nix.conf または configuration.nix
nix.settings.trusted-users = [ "root" "@wheel" ];
```

10.4.2 バイナリキャッシュの署名検証

```
nix.settings = {
  substituters = [
    "https://cache.nixos.org"
    "https://my-cache.example.com"
  ];
  trusted-public-keys = [
    "cache.nixos.org-1:6NCHdD59X431o0gWypbMrAURkbJ16ZPMQFGspcDShjY="
    "my-cache-1:XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX="
  ];
};
```

10.5 プロジェクト構成のベストプラクティス

推奨するプロジェクト構成は以下の通りである。

```
my-project/
├── flake.nix          # Flake定義
├── flake.lock         # ロックファイル（Git管理対象）
├── .envrc             # direnv設定
├── nix/
│   ├── default.nix    # パッケージ定義
│   ├── shell.nix      # 開発環境（Flake以前の互換用）
│   └── overlay.nix    # カスタムオーバーレイ
├── src/               # ソースコード
└── tests/             # テスト
```

10.5.1 .gitignore に追加すべきもの

```
result
result-*
```

`result` は `nix build` が作成するシンボリックリンクであり、Git 管理対象にすべきでない。

10.6 コミュニティリソース

Nix の学習やトラブルシューティングに役立つリソースを紹介する。

- **Nix 公式マニュアル**: nixos.org/manual/nix
- **Nixpkgs マニュアル**: nixos.org/manual/nixpkgs
- **NixOS Wiki**: wiki.nixos.org
- **nix.dev**: Nix の入門チュートリアル
- **NixOS Discourse**: discourse.nixos.org（フォーラム）
- **Nixpkgs GitHub**: github.com/NixOS/nixpkgs
- **Zero to Nix**: zero-to-nix.com（対話的チュートリアル）